



Universidad Don Bosco

FACULTAD DE INGENIERÍA

ESCUELA DE COMPUTACIÓN

Desarrollo de aplicaciones con Software Propietario

Guía 6: Programación Orientada a Objetos (POO)



I. Objetivos

Que el estudiante sea capaz de:

- Defina que es la Programación Orientada a Objetos
- Sea capaz de crear clases en C#.NET
- Defina propiedades y métodos dentro de una clase

II. Introducción Teórica

.NET y la Programación Orientada a Objetos (POO)

El .NET Framework (y su evolución, .NET 5+) está basado en un modelo de clases y objetos.

En C#, prácticamente todas las funcionalidades se implementan a través de clases incluidas en la biblioteca de .NET, lo que ofrece un entorno altamente orientado a objetos sin limitaciones significativas.

Debido a esta arquitectura, C# aprovecha plenamente las características de la Programación Orientada a Objetos (POO), permitiendo herencia, encapsulación y polimorfismo de forma nativa.

Fundamentos de la POO en C#

Los tres pilares fundamentales de la POO son:

1. Herencia

Permite crear clases basadas en otras clases ya existentes. La nueva clase hereda atributos y métodos de la clase base, pudiendo además modificar su comportamiento o añadir nuevas funcionalidades.

Ejemplo:

- Clase base: Persona
- Clases derivadas: Cliente, Empleado

C# admite herencia simple a nivel de clase (una clase solo puede heredar de otra), pero puede implementar múltiples interfaces.

2. Encapsulación

Consiste en ocultar los detalles internos de una clase y exponer únicamente lo necesario mediante una interfaz pública.

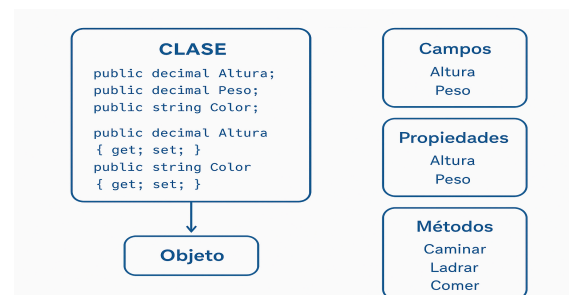
Esto protege los datos y separa la implementación interna del uso externo.

3. Polimorfismo

Permite que diferentes clases definan métodos con el mismo nombre pero con comportamientos distintos, pudiendo ser utilizados de forma uniforme a través de referencias de la clase base.

Definición de una Clase

Una **clase** es un modelo que describe las características (propiedades) y comportamientos (métodos) de un objeto.



Ejemplo básico:

```
class Perro
{
    public decimal Altura;
    public decimal Peso;
    public string Color;

    public void Caminar(int pasos) { }
    public void Ladrar() { }
    public void Comer() { }
}
```

¿Qué es un Objeto?

Un **objeto** es una instancia de una clase.

La clase es el molde, y el objeto es la pieza creada a partir de él.

Ejemplo:

```
Perro pastorAleman = new Perro();
pastorAleman.Color = "Negro";
pastorAleman.Peso = 25;
```

Miembros de una Clase

Los miembros pueden ser:

- **Campos** → variables que almacenan datos de la clase.
- **Propiedades** → encapsulan el acceso a los campos.
- **Métodos** → definen comportamientos.

Ejemplo de Campos

```
Perro pastorAleman = new Perro();
pastorAleman.Color = "Negro";
pastorAleman.Peso = 25;
```

Ejemplo de Propiedades

Las propiedades permiten controlar la lectura y escritura de datos:

```
class Perro
{
    private decimal altura;
    private string color;

    public decimal Altura
    {
        get { return altura; }
        set { altura = value; }
    }

    public string Color
    {
        get { return color; }
        set { color = value; }
    }

    public decimal Peso { get; private set; } // Solo lectura pública
}
```

Propiedades de Solo Lectura y Solo Escritura

- **Solo lectura** → Solo tienen get.
- **Solo escritura** → Solo tienen set.

```

class Empleado
{
    private string _ctaBancaria;
    private string _entidad;
    private DateTime _fechaIngreso;

    public string CtaBancaria
    {
        set
        {
            _ctaBancaria = value;
            _entidad = value switch
            {
                "1111" => "Banco del Perú",
                "2222" => "Banco Continental",
                _ => "Entidad desconocida"
            };
        }
    }

    public DateTime FechaIngreso
    {
        get => _fechaIngreso;
        set => _fechaIngreso = value;
    }

    public string Entidad => _entidad; // Solo Lectura

    public string MesIngreso => _fechaIngreso.ToString("MMM");

    public double CalcularSueldo(int horas, double costoHora) =>
        horas * costoHora;
}

```

Buenas Prácticas

- Usar propiedades automáticas (public string Nombre { get; set; }) cuando no se requiere lógica especial.
- Mantener los campos privados y exponerlos mediante propiedades para preservar la encapsulación.
- Usar nombres en PascalCase para clases, métodos y propiedades, y camelCase para parámetros y campos privados.
- Aprovechar la herencia y las interfaces para mejorar la reutilización y flexibilidad del código.

III. Requerimientos

Nº	Cantidad	Descripción
1	1	PC con Microsoft Visual Studio 2022 .NET o superior, instalado
2	1	Guía 6
3	1	Memoria USB

IV. Procedimiento

Registran información personal de un empleado.

Para comenzar el desarrollo de tu proyecto, se requiere crear una clase que permita manejar los datos personales y laborales de un empleado general. Esta clase debe incluir:

- Datos personales: nombre, edad, etc.
- Datos laborales: fecha de contrato, sueldo base.
- Restricciones: la edad del empleado debe estar entre 18 y 50 años.
- Cálculos: se aplicarán tasas de renta e ISSS, proporcionadas por un administrador.

1. Crear Clase clsEmpleado

1. Crear un nuevo proyecto tipo Windows form
2. En el **Explorador de soluciones**, haz clic derecho sobre el nombre del proyecto.
3. Selecciona **Agregar** → **Clase...**
4. Nombra el nuevo módulo de clase como clsEmpleado.

clsEmpleado

```
public class clsEmpleado
{
    // ----- Atributos privados -----
    private string Nombres;
    private string Apellidos;
    private DateTime FechaNacimiento;
    private int Edad;

    private DateTime FechaContratacion;
    private decimal Tasalss;
    private decimal TasaRenta;
    private decimal SueldoBase;
    private decimal SueldoFinal;

    private bool DatosCompleto;
    private bool DatosLaboralesListos;

    private string ultimoError = string.Empty;

    // ----- Propiedades públicas (compatibilidad) -----
    /// <summary>Apellido, Nombres</summary>
```

```
public string nombrecompleto => $"{Apellidos}, {Nombres}";

/// <summary>Indica si los datos personales pasaron validación</summary>
public bool datospersonales_aceptados => DatosCompletos;

/// <summary>Indica si los datos laborales pasaron validación</summary>
public bool datoslaborales_aceptados => DatosLaboralesListos;

/// <summary>Mensaje de error/resultados de la última validación</summary>
public string UltimoError => ultimoError;

// ----- Constructor -----
public clsEmpleado()
{
    TasaIss = 3.0m; // valor por defecto (porcentual)
    TasaRenta = 10.0m; // valor por defecto (porcentual)
    FechaContratacion = DateTime.Now;
    DatosCompletos = false;
    DatosLaboralesListos = false;
}

// ----- Métodos de validación y asignación -----
/// <summary>
/// Valida y asigna los datos personales.
/// Devuelve true si OK, false y UltimoError en caso contrario.
/// </summary>
public bool DefinirDatosPersonales(string nom, string apel, DateTime fechanac)
{
    ultimoError = string.Empty;
    DatosCompletos = false;

    if (string.IsNullOrEmpty(nom))
    {
        ultimoError = "Falta ingresar nombres del empleado.";
        return false;
    }
    if (string.IsNullOrEmpty(apel))
    {
        ultimoError = "Falta ingresar apellidos del empleado.";
        return false;
    }
}
```

```
Nombres = nom.Trim();
Apellidos = apel.Trim();

// Cálculo de edad preciso
int edadCalculada = DateTime.Now.Year - fechanac.Year;
if (fechanac > DateTime.Now.AddYears(-edadCalculada)) edadCalculada--;

if (edadCalculada < 0)
{
    ultimoError = "Fecha de nacimiento inválida.";
    return false;
}
if (edadCalculada < 18)
{
    ultimoError = "Persona menor de edad según código de trabajo (mínimo 18 años).";
    return false;
}
if (edadCalculada > 50)
{
    ultimoError = "Empleado excede edad máxima permitida (según regla del ejercicio).";
    return false;
}

FechaNacimiento = fechanac;
Edad = edadCalculada;
DatosCompleto = true;
return true;
}

/// <summary>
/// Valida y asigna datos laborales (fecha contrato y sueldo).
/// Devuelve true si OK y deja UltimoError si no.
/// </summary>
public bool DefinirDatosLaborales(DateTime fechacontrato, decimal sueldoinic)
{
    ultimoError = string.Empty;
    DatosLaboralesListos = false;

    if (!DatosCompleto)
    {
```

```
        ultimoError = "Debe completar primero los datos personales.";
        return false;
    }

    if (sueldoinic < 0)
    {
        ultimoError = "Sueldo inicial no puede ser negativo.";
        return false;
    }

    DateTime fechaMinLaboral = FechaNacimiento.AddYears(18);
    DateTime fechaHoy = DateTime.Now;

    if (fechacontrato < fechaMinLaboral)
    {
        ultimoError = $"Fecha de contrato inválida. Debe ser >= {fechaMinLaboral:d}.";
        return false;
    }
    if (fechacontrato > fechaHoy)
    {
        ultimoError = $"Fecha de contrato inválida. No puede ser futura (hoy: {fechaHoy:d}).";
        return false;
    }

    FechaContratacion = fechacontrato;
    SueldoBase = Math.Round(sueldoinic, 2);
    DatosLaboralesListos = true;

    CalcularSueldoNeto();
    return true;
}

/// <summary>
/// Asigna las tasas (porcentaje) para ISSS y Renta. Recalcula sueldo.
/// Rango razonable: 0..40 (%)
/// </summary>
public void AsignarDescuentos(decimal iss = 2.50m, decimal renta = 10.50m)
{
    if (iss >= 0m && iss <= 40m) TasaIss = iss;
    if (renta >= 0m && renta <= 40m) TasaRenta = renta;
    CalcularSueldoNeto();
}
```



```

    }

    /// <summary>
    /// Retorna sueldos como cadenas formateadas (compatibilidad con código existente)
    /// </summary>
    public void VerSueldos(ref string sb, ref string sf)
    {
        sb = SueldoBase.ToString("F2", CultureInfo.CurrentCulture);
        sf = SueldoFinal.ToString("F2", CultureInfo.CurrentCulture);
    }

    // ----- Método interno -----
    private void CalcularSueldoNeto()
    {
        decimal descIcss = SueldoBase * (TasaIcss / 100m);
        decimal descRenta = SueldoBase * (TasaRenta / 100m);
        SueldoFinal = Math.Round(SueldoBase - descIcss - descRenta, 2);
    }

    // Para debugging/registro si se requiere
    public override string ToString() => $"{nombrecompleto} - Sueldo base: {SueldoBase:F2} -
    Neto: {SueldoFinal:F2}";
}

```

2. Crear Clase clsPlanilla

1. En el **Explorador de soluciones**, haz clic derecho sobre el nombre del proyecto.
2. Selecciona **Agregar** → **Clase...**
3. Nombra el nuevo módulo de clase como clsPlanilla.

clsEmpleado

```

public class clsPlanilla
{
    private int Estado; // 1: Sin iniciar, 2: En proceso, 3: Finalizada
    private string Empresa;
    private DateTime FechaPlanilla;

    private Dictionary<int, clsEmpleado> Listaempleados = new Dictionary<int, clsEmpleado>();
    private int TotalEmpleados;

    /// <summary>
    /// Si true, la clase mostrará mensajes vía MessageBox. Si false, solo emitirá el evento Notificar.

```

```

/// Mejor práctica: suscribirse al evento Notificar en la UI y poner MostrarMensajes = false.
/// </summary>
public bool MostrarMensajes { get; set; } = true;

/// <summary>
/// Evento que notifica mensajes; la UI puede suscribirse para recibirlos.
/// </summary>
public event Action<string, string> Notificar;

public clsPlanilla()
{
    Estado = 1;
    Empresa = string.Empty;
    FechaPlanilla = DateTime.Now;
    TotalEmpleados = 0;
}

// Método para centralizar notificaciones
private void EnviarMensaje(string mensaje, string titulo = "")
{
    Notificar?.Invoke(titulo, mensaje);
    if (MostrarMensajes)
    {
        MessageBox.Show(mensaje, string.IsNullOrEmpty(titulo) ? "Información" : titulo);
    }
}

public void AbrirPlanilla(DateTime fechaInicio, string nombreempresa = "(Sin nombre)")
{
    nombreempresa = (nombreempresa ?? string.Empty).Trim();
    switch (Estado)
    {
        case 1:
            FechaPlanilla = fechaInicio;
            if (nombreempresa.Length > 0) Empresa = nombreempresa;
            Estado = 2;
            EnviarMensaje("Planilla Abierta, inicie registro empleados",
                $"Planilla de {Empresa}, APERTURA: {FechaPlanilla}");
            break;

        case 2:
    }
}

```

```
        EnviarMensaje($"Planilla ya está abierta desde el: {FechaPlanilla}", $"Planilla de {Empresa}");
        break;

    case 3:
        EnviarMensaje($"Planilla creada el {FechaPlanilla} ya se cerró", $"Planilla de {Empresa}");
        break;
    }
}

public void RecibirEmpleado(clsEmpleado nuevoEmpleado)
{
    if (nuevoEmpleado == null)
    {
        EnviarMensaje("Empleado nulo. No se puede registrar.", "Error");
        return;
    }

    if (Estado != 2)
    {
        EnviarMensaje("Planilla aún no está abierta", $"Planilla de {Empresa}");
        return;
    }

    if (!nuevoEmpleado.datospersonales_aceptados)
    {
        EnviarMensaje("Error, datos personales están incompletos", "Control planilla");
        return;
    }

    if (!nuevoEmpleado.datoslaborales_aceptados)
    {
        EnviarMensaje("Error, datos laborales están incompletos", "Control planilla");
        return;
    }

    TotalEmpleados++;
    Listaempleados.Add(TotalEmpleados, nuevoEmpleado);
}

public void GenerarListado(ref DataGridView cuadro)
{
    if (cuadro == null) throw new ArgumentNullException(nameof(cuadro));
}
```

```
switch (Estado)
{
    case 1:
        EnviarMensaje("Planilla aún no ha sido abierta", $"Planilla {Empresa}");
        return;

    case 2:
        if (TotalEmpleados == 0)
        {
            EnviarMensaje("Planilla no tiene aún empleados registrados", $"Planilla de {Empresa}");
            return;
        }
        Estado = 3;
        EnviarMensaje($"Planilla cerrada con {TotalEmpleados} empleados", $"Planilla de {Empresa}");
        EnviarMensaje($"Planilla abierta el {FechaPlanilla} se muestra ahora!!", $"Planilla de
{Empresa}");
        break;
}

// Preparar DataGridView
cuadro.SuspendLayout();
cuadro.Rows.Clear();
cuadro.Columns.Clear();

// Columnas
cuadro.Columns.Add("id", "ID");
cuadro.Columns.Add("nom", "Nombre completo");
cuadro.Columns.Add("sb", "Sueldo base");
cuadro.Columns.Add("sf", "Sueldo neto final");

// Formatos y comportamiento
cuadro.ReadOnly = true;
cuadro.AllowUserToAddRows = false;
cuadro.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill;

// Estilos
cuadro.RowsDefaultCellStyle.BackColor = System.Drawing.Color.Bisque;
cuadro.AlternatingRowsDefaultCellStyle.BackColor = System.Drawing.Color.Beige;

// Llenar filas
```

```
int i = 0;
foreach (var kvp in Listaempleados.OrderBy(k => k.Key))
{
    var empleado = kvp.Value;
    string sb = "0";
    string sf = "0";
    empleado.VerSueldos(ref sb, ref sf);

    cuadro.Rows.Add();
    cuadro.Rows[i].Cells[0].Value = kvp.Key;
    cuadro.Rows[i].Cells[1].Value = empleado.nombrecompleto;
    cuadro.Rows[i].Cells[2].Value = sb;
    cuadro.Rows[i].Cells[3].Value = sf;
    i++;
}

cuadro.ResumeLayout();
EnviarMensaje("Planilla de pago final completa generada en pantalla!!");
}

/// <summary>Propiedad para obtener total de empleados como cadena (compatibilidad)</summary>
public string TotaldeEmpleado => TotalEmpleados.ToString();

/// <summary>Acceso de solo lectura a los empleados (si necesario para otras
operaciones)</summary>
public IReadOnlyDictionary<int, clsEmpleado> Empleados => new
System.Collections.ObjectModel.ReadOnlyDictionary<int, clsEmpleado>(Listaempleados);
}
```

Integrar Clase a Formulario

1. Ahora diseñe un form con los siguientes controles mostrados en la imagen.
2. Proceda a modificar las propiedades de los controles anteriores, de acuerdo a la tabla de asignaciones a continuación:

Form1

Datos de la Planilla

Fecha Planilla: 2025-08-12

Empresa:

Abrir Planilla

Datos del Empleado

Nombres:

Apellidos:

Fecha Nacimiento: 2025-08-12

Fecha Contrato: 2025-08-12

Sueldo:

Agregar Empleado

Planilla de Empleados

Mostrar Planilla

Tipo de Control	Nombre del Control	Propiedades a Modificar
GroupBox	grpPlanilla	Text = "Datos de la Planilla"
TextBox	txtFechaPlanilla	Text ="2025-08-12"
TextBox	txtEmpresa	
Button	btnAbrirPlanilla	Text = "Abrir Planilla"
GroupBox	grpEmpleado	Text = "Datos del Empleado"
TextBox	txtNombres	
TextBox	txtApellidos	

TextBox	txtFechaNacimiento	2025-08-12
TextBox	txtFechaContrato	2025-08-12
TextBox	txtSueldoBase	
Button	btnAgregarEmpleado	Text = "Agregar Empleado"
GroupBox	grpMostrarPlanilla	Text = "Planilla de Empleados"
Button	btnMostrarPlanilla	Text = "Mostrar Planilla"
DataGridView	dataGridViewEmpleados	

1. Instanciar de forma global el objeto **planilla**, con la siguiente línea de código:

```
private clsPlanilla planilla = new clsPlanilla();
```

3. Ahora, digite los siguientes bloques de código dentro de c/procedimiento de evento indicado

Control	Evento
Form1	
<pre>InitializeComponent(); // Configurar planilla para que no muestre mensajes automáticos, usaremos evento planilla.MostrarMensajes = false; planilla.Notificar += Planilla_Notificar; // Configurar DataGridView (opcional, si no configuras en diseñador) dataGridViewEmpleados.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill; dataGridViewEmpleados.AllowUserToAddRows = false; dataGridViewEmpleados.ReadOnly = true; dataGridViewEmpleados.Columns.Add("id", "ID"); dataGridViewEmpleados.Columns.Add("nom", "Nombre completo"); dataGridViewEmpleados.Columns.Add("sb", "Sueldo base"); dataGridViewEmpleados.Columns.Add("sf", "Sueldo neto final");</pre>	
btnAbrirPlanilla	Click
<pre>DateTime fecha; if (!DateTime.TryParse(txtFechaPlanilla.Text, out fecha)) { MessageBox.Show("Fecha inválida para la planilla"); return; } string empresa = txtEmpresa.Text.Trim(); planilla.AbrirPlanilla(fecha, empresa);</pre>	

btnAgregarEmpleado	Click
	<pre>// Validar datos personales string nombres = txtNombres.Text.Trim(); string apellidos = txtApellidos.Text.Trim(); DateTime fechaNac; if (!DateTime.TryParse(txtFechaNacimiento.Text, out fechaNac)) { MessageBox.Show("Fecha de nacimiento inválida"); return; } // Validar datos laborales DateTime fechaContrato; if (!DateTime.TryParse(txtFechaContrato.Text, out fechaContrato)) { MessageBox.Show("Fecha de contrato inválida"); return; } decimal sueldoBase; if (!decimal.TryParse(txtSueldoBase.Text, NumberStyles.Currency, CultureInfo.CurrentCulture, out sueldoBase)) { MessageBox.Show("Sueldo base inválido"); return; } var empleado = new clsEmpleado(); if (!empleado.DefinirDatosPersonales(nombres, apellidos, fechaNac)) { MessageBox.Show(\$"Error datos personales: {empleado.UltimoError}"); return; } if (!empleado.DefinirDatosLaborales(fechaContrato, sueldoBase)) { MessageBox.Show(\$"Error datos laborales: {empleado.UltimoError}"); return; } planilla.RecibirEmpleado(empleado); MessageBox.Show("Empleado agregado correctamente");</pre>
btnMostrarPlanilla	Click
	<pre>planilla.GenerarListado(ref dataGridViewEmpleados);</pre>

planillaEPSON.GenerarListado(ref dataGridView1);
Planilla_Notificar(string titulo, string mensaje) Funcion propia
MessageBox.Show(mensaje, string.IsNullOrEmpty(titulo) ? "Información" : titulo);

1. Ejecute de nuevo el proyecto y presione el botón btnMostrarPlanilla. Evalúe el mensaje mostrado. No finalice la ejecución, para realizar las siguientes pruebas:
 - presione botón btnAbrirPlanilla y luego presione nuevamente a btnMostrarPlanilla.
 - Evalúe los mensajes y resultados generados.
 - Ingrese unos 2 o 3 empleados con datos correctos y luego presione otra vez botón btnMostrarPlanilla.
 - Evalúe los mensajes y resultados generados.
2. Finalmente, ejecute la aplicación, ingrese 3 o 4 empleados con datos correctos y genere la planilla.

V. Ejercicios complementarios

El instructor evaluará el desempeño de su trabajo dentro la clase. Estos ejercicios se evaluarán en la misma hora de clase.

1. Dentro de la clase clsPlanilla, desarrolle un método que sea capaz de generar un archivo de texto simple/plano (.txt, cuyo nombre lo definirá usuario desde un parametro). Este archivo de texto tendra a los datos generados en la planilla.
2. El archivo se generara solamente si ya se imprimió la planilla en el datagridview.

VI. Análisis de resultados

Para la próxima semana

1. Crea una clase llamada **Cuenta** que permita administrar la información de una cuenta bancaria. Esta clase debe incluir los siguientes atributos:
 - Número de cuenta
 - DUI del cliente
 - Saldo actual
 - Interés anual aplicado a la cuenta (con un rango permitido entre 10% y 22.3%).

El número de cuenta debe seguir el formato **DDMMYYYY-XXXXXX**, donde **DDMMYYYY** corresponde a la fecha (día, mes, año) y **XXXXXX** es un número correlativo que comienza en **1000001**. Define en la clase los siguientes miembros:

- Propiedades para el número de cuenta, el DUI y el saldo actual.
- Método actualizarSaldo(): actualizará el saldo aplicando el interés diario, calculado como el interés anual dividido entre 365, aplicado sobre el saldo actual.
- Método ingresar(double cantidad): permitirá depositar una cantidad de dinero en la cuenta.

- Método retirar(double cantidad): permitirá retirar una cantidad de dinero, siempre que exista saldo suficiente.
- Método para mostrar todos los datos de la cuenta de manera clara y completa.

Con esta clase, desarrolla una aplicación para el Banco Adajet que gestione el control de **n** cuentas de ahorro de sus clientes.

Considera que un cliente puede decidir cerrar su cuenta, por lo que la aplicación debe manejar esa opción.

Finalmente, asegúrate de implementar las validaciones necesarias dentro de la clase **Cuenta** para garantizar la integridad y consistencia de los datos.

V. Bibliografía

1. Microsoft Learn - Documentación oficial de .NET, <https://learn.microsoft.com/es-es/dotnet/> Incluye tutoriales de C#, WinForms, y controles UI.
2. Troelsen, A. (2023). Pro C# 10 with .NET 6. Apress. Libro de referencia para sintaxis avanzada, patrones de diseño y buenas prácticas.
3. Richter, J. (2023). CLR via C#. Microsoft Press. Fundamentos del runtime de .NET y gestión de memoria.
4. Stack Overflow - Comunidad .NET, <https://stackoverflow.com/questions/tagged/c%23>. Resolución de errores comunes y mejores prácticas validadas por la comunidad.
5. DevExpress - WinForms UI Best Practices, <https://www.devexpress.com/winforms/> Guías de diseño de interfaces y uso de controles avanzados.

Recursos adicionales y Lecturas adicionales

- **Extensión de Visual Studio:**
[Microsoft XAML Controls Gallery](#) para inspirar diseños modernos.
- **Video-tutoriales:**
Playlist de YouTube sobre WinForms (ej: "Canal de Programming with Mosh").
- **Libros:**
 - Troelsen, A. (2022). Pro C# 10 with .NET 6. Apress.
 - Richter, J. (2020). CLR via C#. Microsoft Press.
- Medrano, K., Hernandez, E., Tejada, R., Gonzalez, B., & Fuentes, N. (2023, July 21). Design of a Prototype System for Real-Time Water Quality Monitoring in Tilapia Farms Using IoT Technology. Presented at the 21st LACCEI International Multi-Conference for Engineering, Education and Technology (LACCEI 2023). <https://doi.org/10.18687/laccei2023.1.1.316>. Disponible en: <http://hdl.handle.net/11715/2757>.
- Medrano, K., Tejada, R., Gonzalez, B., & Juarez, S. (2023, December 10). Aplicación Móvil Interactiva de Realidad Aumentada para la Interacción de Ambientes Culturales. Presented at the 1ra Exposición de Pósteres Científicos y Conferencias Tecnológicas, Universidad Don Bosco, El Salvador. Disponible en: <http://hdl.handle.net/11715/2759>.